

INSTRUKCJA DO PROJEKTU

Optymalizacja struktur i obliczeń w sieciach neuronowych

Dr hab. Inż. Piotr Szczuko, prof. Politechniki Gdańskiej



Wprowadzenie

Dokument ten stanowi pierwszą część instrukcji projektowej, opisuje wymagania i wytyczne realizacji etapów 1-7, które dotyczą zdefiniowania zadania do realizacji, wyboru danych, modelu, ustalenia procedury optymalizacji modelu i jego uczenia oraz testowania i porównania z cechami modelu referencyjnego – bazowego.

Projekt powinien być wykonany jako kod w języku Python, dokumentacja powinna uwzględniać etapy przygotowania środowiska (instalacja bibliotek, używane wersje), pobierania danych, modyfikacji danych, pobierania modelu, optymalizacji i badania. Kod powinien być odpowiednio czytelnie komentowany oraz powinno być możliwe jego uruchomienie bez błędów – tzn. wszystkie zależności powinny być spełnione, używane ścieżki poprawne, itd.

Zawartość instrukcji podaje wytyczne do realizacji oraz aspekty i elementy rozwiązania wymagające pisemnego udokumentowania.

Przygotowanie danych i modelu (etapy 1-3)

Wybór zagadnienia do realizacji

Wybierając zadanie do realizacji, rozważyć należy możliwe problemy klasyfikacji złożonych danych wielowymiarowych. Sugerowana jest:

- analiza obrazu lub dźwięku w postaci mel-spektrogramu (obrazu kolorowego)
- oraz postawienie sobie za cel rozpoznawania nowych klas, które nie były wcześniej określone dla wykorzystywanego zbioru danych.

Dane takie w praktycznym wykorzystaniu nastroczają wiele problemów, m.in. trudność w pozyskiwaniu, poprawnym opisywaniu przed etapami treningu i walidacji, podatność dokładności modelu na szum danych i etykiet, złożoność algorytmów, które osiągałyby odpowiednio wysoką dokładność i inne.

Z tego powodu są to dane, dla których użycie metod optymalizacji obliczeń, oczyszczania, optymalizacji treningu i wnioskowania dają istotne i obserwowalne rezultaty:

- zwiększenie dokładności (wykazane poprzez pomiary zmian wybranej metryki),
- skrócenie czasu obliczeń (epok uczenia, czasu wnioskowania),
- zmniejszenie rozmiaru modelu (liczba parametrów, wielkość wynikowego pliku z modelem).

Wskazane jest, aby te kryteria poddawać ocenie, obiektywnemu pomiarowi i analizie porównawczej pomiędzy podejściem bez optymalizacji a optymalizowanym.

Celem projektu jest praktyczne wykorzystanie zagadnień prezentowanych w ramach wykładów z przedmiotu oraz zaadoptowanie podejść użytych w trakcie laboratoriów, do realizacji własnego celu. Rezultat należy ocenić pod kątem uzyskanego wzrostu dokładności oraz

wydajności, w porównaniu do podejścia opartego wyłącznie na dotrenowaniu gotowego modelu do nowych klas, określonych przez autora projektu.

Dokumentacja etapu 1

W projekcie wymagane jest odpowiednie udokumentowanie i opisanie zagadnienia wybranego do realizacji przez autora projektu:

- słowny opis celu,
- uzasadnienie wyboru celu (dlaczego jest on istotny, jak może być zastosowany docelowo w aplikacji),
- opisanie czego oczekuje się od optymalizacji w tym zagadnieniu (np. dostosowania do ograniczonych zasobów pewnej wybranej platformy).

2. Wybór danych i modelu

Określone powyżej zadanie: dotrenowanie modelu do nowych klas, wymaga posiadania zbioru danych i gotowego modelu klasyfikującego obiekty z tego zbioru. Mogą być to zbiór obrazów z 1000 klas: ImageNET (<https://image-net.org/>) lub 100 klas CIFAR-100 (<https://www.cs.toronto.edu/~kriz/cifar.html>) i wstępnie wytrenowany klasyfikator oparty na np. MobileNet. Wykorzystać można inne dowolne dane, po wstępnej konsultacji z prowadzącym przedmiot.

W tym projekcie jest wskazane wybranie modelu, który nie jest optymalizowany – np. nie używano upraszczania modelu, ani technik destylacji lub podobnych. To zadaniem autora projektu jest takie dostosowanie modelu i jego zoptymalizowanie, aby w mierzalnym stopniu były do zaobserwowania (udokumentowania) korzystne rezultaty tych operacji.

Dokumentacja etapu 2

W dokumentacji projektu zawrzeć należy opis następujących aspektów i decyzji projektowych podejmowanych przez autora:

- podanie źródeł danych i modelu,
- uzasadnienie wyboru danych i modelu,
- scharakteryzowanie oryginalnych danych: rozmiary, liczba klas, format danych, itd.,
- scharakteryzowanie oryginalnego modelu: rozmiar, liczba parametrów, czas wnioskowania, czas treningu pojedynczej epoki, wymagania pamięciowe (samodzielnie wykonać test uczenia jednej epoki na podzbiorze minibach o zadanym rozmiarze – zaraportowanie zajętości pamięci i czasu treningu),
- krótkie porównanie z innymi dostępnymi zbiorami, podobnymi pod względem dziedziny (nie ma konieczności uruchamiania treningu, testu. Odwołać można się do porównań dostępnych w publikacjach).

3. Określenie zadania docelowego i przygotowanie danych

Krokiem kolejnym jest **określenie nowych klas**, które może polegać na wyborze podzbioru z obecnych klas i dodaniu kilku nowych klas lub utworzeniu nowych klas na bazie oryginalnych. Np. z istniejących klas zwierząt: bez zmian zachować klasy kilku ssaków, a wszystkie klasy gatunków gadów złączyć w jedną klasę „reptiles”, itp., co nazywane jest **przemapowaniem** klas. Możliwe jest także utworzenie klas zupełnie od podstaw, częściowo w oparciu o istniejący zbiór opcjonalnie uzupełniany własnymi obrazami.

Zalecane jest w zbiorze nowych klas (zwłaszcza jeśli jest on wynikiem prostego przemapowania) **celowo wprowadzić przekłamania** w wartościach etykiet. W ten sposób możliwe będzie wykazanie różnic między rezultatami douczania oraz douczania z optymalizacjami, wyjaśnianymi w ramach niniejszego przedmiotu.

Dokumentacja etapu 3

Dokumentacja powinna zawierać opisane:

- dane dla nowego zadania - lista rozpoznawanych klas, z krótkim uzasadnieniem, dlaczego określony na początku cel wymaga takich klas,
- sposób przygotowania danych: jakie klasy łączono, usuwano, zmieniono nazwy, dodano ręcznie, jakie nowe próbki nowych klas dostarczono i opisano samodzielnie, jak prowadzono procedurę zaszumiania etykiet,
- **uwaga:** wszystkie operacje realizowane w sposób losowy (np. szum) powinny w kodzie wykorzystywać ręczne zadawanie ziarna dla generatora liczb pseudo-losowych, aby zapewnić powtarzalność/replikowalność wyników.
- **uwaga:** w tym etapie można wykorzystać wiedzę zdobytą podczas realizacji przedmiotu „Zaawansowane przygotowanie danych w uczeniu maszynowym”, jednak każdy etap odpowiednio udokumentować, bez stosowania „skrótów myślowych”, w sposób taki, aby nie było potrzeby odwoływania się do materiałów z tego przedmiotu.

Pomiary modelu odniesienia (etap 4)

Proces douczania w tym etapie należy wykonać dla oryginalnego modelu. Poprzez takie douczenie bez optymalizacji uzyskane zostaną dane, które wykorzystać należy w dalszych etapach pracy do oceny modelu optymalizowanego.

4. Douczenie modelu odniesienia (baseline)

Typowo w prostych podejściach, które nie mają na celu optymalizacji a zapewnić mają dostosowanie do nowego zagadnienia (ang. Transfer learning, transfer wiedzy), stosuje się douczenie oryginalnego modelu nowymi danymi, zwykle bez zmiany jego architektury. Adaptacja

wymaga zawsze zmiany liczby neuronów w warstwie wyjściowej, odpowiednio do liczby nowych klas oraz ustawienia bardzo niskiego lub zerowego tempa nauki dla wczesnych warstw w modelu i tym samym istotne douczanie warstw głębszych i wyjściowych.

Dokumentacja etapu 4

W dokumentacji projektu zamieścić należy kod z komentarzem w jaki sposób model oryginalny douczony został do nowego zadania oraz jakie uzyskano miary. Zostaną one porównane z analogicznymi wartościami dla modelu optymalizowanego.

Miary odniesienia to przykładowo:

- czas uczenia pojedynczej epoki,
- uzyskaną dokładność klasyfikacji (np. F1-score multiclass classification),
- rozmiar modelu, wymagania pamięciowe,
- inne.

Uwaga: samodzielnie zdecydować trzeba o stosowanych funkcjach straty (ang. Loss), optymalizacji treningu i innych hiperparametrach modelu, odpowiednio zamieszczając w dokumentacji ich opis.

Optymalizacja modelu (etapy 5-6)

5. Upraszczenie modelu

Jedną z popularnych metod optymalizacji modelu jest jego przeredzanie (patrz: wykład z przedmiotu OSIOSN). Polega ono na wyzerowaniu w procesie uczenia tych wag, które osiągają wartości odpowiednio bliskie zeru. Popularne biblioteki języka Python wspierające uczenie sieci neuronowych (PyTorch, Tensorflow, MXNet i ich pochodne, np. Keras, Sonnet, Gluon) oferują odpowiednie metody przeredzania (ang. Sparsification), z których przykłady prezentowane były na laboratorium.

Zalecane jest przypomnienie sobie informacji teoretycznych i zadań praktycznych (wykłady i laboratoria z przedmiotu). Następnie należy samodzielnie dokonać przeglądu materiałów podanych jako referencje w ramach wykładu, podjąć decyzję odnośnie podejścia, wybrać implementację i ją zastosować. W ramach wykładu przedstawiono strategie i główne nurty, z których wybrać i zastosować należy co najmniej jeden:

- Model sparsification for inference – przeredzanie modelu po wytrenowaniu, przed jego użyciem
- Model sparsification for training – przeredzanie modelu w trakcie treningu, modyfikacje w trakcie obliczeń wstecznej probagacji błędów (ang. Backpropagation),
- Ephemeral sparsification – przeredzanie na podstawie instancji danych.

Dokumentacja etapu 5

W dokumentacji projektowej należy:

- opisać wybrany wariant upraszczania, przerzedzania modelu, zastosowany w swoim projekcie,
- uzasadnić należy wybór wariantu,
- skomentować, dlaczego podejście jest dobrze dostosowane do postawionego celu,
- z dokumentacji użytej metody wybrać i zamieścić krótki opis adekwatnych cech: jaka jest zastosowana funkcja straty, regularyzacja, parametry metody upraszczania,
- jeśli upraszczanie stosuje scenariusz (ang. Schedule) to należy go krótko opisać: np. podając dane na temat etapów trenuj-upraszczaj, lub trenuj-upraszczaj-potwórz, lub trenuj-upraszczaj-dodawaj (ang. Regrowth)-potwórz.
- rozważyć możliwość zastosowania algorytmu RigL (odpowiednie źródła, publikacje i kod dostępne na: <https://paperswithcode.com/paper/rigging-the-lottery-making-all-tickets-1>, <https://arxiv.org/abs/2103.15767> i <https://arxiv.org/abs/1911.11134>. Jeśli algorytm nie zostanie zastosowany, należy krótko uzasadnić, dlaczego nie było możliwości jego użycia.

6. Dodatkowe optymalizacje modelu

Opcjonalnie, odpowiednio do możliwości modelu i charakteru rozwiązywanego problemu, rozważyć należy możliwość zastosowania zmiany precyzji w modelu, kwantyzacji wag, kompresji modelu, współdzielenia wag. Jeśli wybrana biblioteka wspiera w prosty sposób te operacje, należy zastosować co najmniej jedną z nich. Proponowane jest zapoznanie się z przykładem: <https://github.com/songhan/Deep-Compression-AlexNet>, gdzie model zmniejszono z 233MB do 8,9MB bez spadku dokładności [8].

Dokumentacja etapu 6

W dokumentacji projektowej należy:

- opisać i przedstawić kod wykorzystanych optymalizacji,
- udokumentować zmianę wielkości modelu i skrócenie czasu treningu jednej epoki na określonym minibatch oraz czas wnioskowania.
- po każdym uproszczeniu modelu, jego kompresji lub użyciu innej techniki należy przeprowadzić test i określić metryki dokładności modelu.
- przedstawić należy porównanie działania modelu odniesienia z modelem optymalizowanym

Jeśli wykorzystywany model posiada warstwy splotowe, to należy skomentować i oszacować korzyści z zastosowania współdzielonych wag filtrów (ang. Kernels) w porównaniu do wariantu,

w którym wykorzystywane byłyby wyłącznie pełne połączenia (ang. Dense, Fully-connected) między warstwami.

Optymalizacja uczenia (etap 7)

Zastosowanie odpowiednio dobranych technik uczenia modeli (szczególnie modeli głębokich) pozwala skrócić proces treningu i zapobiegać przetrenowaniu (ang. Overfitting).

7. Hiperparametry uczenia

Spośród możliwości wybranego frameworku należy wymienić, krótko opisać oraz zastosować kilka podejść (adekwatnie do wykorzystywanego modelu):

- augmentacja danych,
- regularyzacja,
- funkcje aktywacji o prostej do obliczenia pochodnej (np. ReLU, ReLU6),
- korekty wag (odpowiedni, wydajny optimizer zamiast tradycyjnego SGD),
- dropout,
- normalizacja (batch normalization).

Dokumentacja etapu 7

Posiłkując się materiałami z wykładu, krótko opisać jakie są cele zastosowania i oczekiwane korzyści z używania takich metod, a następnie przebadать i udokumentować rezultaty:

- w jaki sposób użycie augmentacji wpłynęło na dokładność modelu?
- jak przebiegała augmentacja danych w tym przypadku, jakie operacje zostały użyte, uzasadnić
 - **uwaga:** nie zawsze rozmycie gaussowskie, obrót, odbicie, itp. operacje stosowane dla grafiki są uzasadnione – np. dla obrazów mel-spektrogramów stosować tylko przesunięcie w poziomie, bez rozmycia, bez rotacji, bez przesunięcia w pionie, bez odbicia.
- jaki i dlaczego wykorzystano sposób regularyzacji,
- jakie konfiguracje hiperparametrów sprawdzono (możliwość zastosowania poszukiwań „grid search”).
- uzasadnić pozostałe podjęte decyzje projektowe i ich wpływ na czasy uczenia, wnioskowania i dokładność, w porównaniu do modelu odniesienia (baseline).
- czy model uproszczony i zoptymalizowany działa istotnie szybciej od modelu odniesienia na CPU? Na GPU? Uwaga: przykłady z literatury pokazują trudności w uzyskaniu przyspieszenia jednocześnie z uproszczeniem, ze względu na niedostosowanie architektury i kodu używanych bibliotek do przetwarzania macierzy rzadkich, jakimi są macierze wag po uproszczeniu (wyzerowaniu wielu wartości).

Literatura uzupełniająca

- [1] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," arXiv:1510.00149, 2015
- [2] Frankle J, Carbin M, The Lottery Ticket Hypothesis; Finding Sparse, Trainable Neural Networks, ICLR 2019, arxiv:1803.03635
- [3] Evci U, Gale T, et al. Rigging the lottery Making All Tickets Winners ICML 2020, arxiv:1911.11134
- [4] Sanh V, Wolf T, Rush A. Movement Pruning: Adaptive Sparsity by Fine-Tuning. (2020). arXiv:2005.07683
- [5] Han S, Kang J. et al., ESE: Efficient Speech Recognition Engine with Sparse LSTM on FPGA. (2017). arXiv:1612.00694
- [6] Chetlur S et al. cuDNN: Efficient primitives for deep learning. (2014). arXiv:1410.0759
- [7] Dave S, Baghdadi R, Nowatzki T, et al. Hardware Acceleration of Sparse and Irregular Tensor Computations of ML Models; A Survey and Insights, 2021, arxiv:2007.00864
- [8] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," arXiv:1510.00149, 2015
- [9] Yu J, Lukefahr A. et al. Scalpel: Customizing dnn pruning to the underlying hardware parallelism. ACM SIGARCH Computer Architecture News 45, 2 (2017), 548–560.
- [10] P. Szczuko, materiały do wykładu Optymalizacja struktur i obliczeń w sieciach neuronowych: Politechnika Gdańska 2022: enauczenie.pg.edu.pl/moodle/course/view.php?id=22383